

Complete time: 30/09/2026.

Q1: The Perceptron Update Geometry

Question: Write the Perceptron Update Rule and explain its geometric interpretation when a misclassification occurs

Answer:

- The update rule is $w \leftarrow w + \eta(y - \hat{y})x$. Geometrically the weight vector acts as the normal vector orthogonal to the decision boundary hyperplane. It strictly updates only when a hard misclassification occurs.

- For example. If a positive sample ($y = 1$) is incorrectly predicted as negative ($\hat{y} = -1$), the angle between w and x is obtuse ($w^T x < 0$). The update adds a fraction of x (specifically $+2\eta x$) to w effectively rotating the decision boundary ~~the~~ normal vector towards x , so it will be correctly classified as positive in future epochs.

- Counterexample: If the dataset contains overlapping classes due to noise (meaning it is not ~~definitely~~ linearly separable), the Perceptron alg will oscillate indefinitely. Because ~~it~~ it relies entirely on direct misclassification to trigger updates, it can never converge to a stabilized "best-fit" line; the normal vector w will perpetually bounce around.

Q2: Linear separability and the XOR Limitation

Question: Explain why a single-layer perceptron fails to learn the XOR function.

Answer:

- A single-layer perceptron computes a single linear combination of its inputs ($w^T x + b$) and applies a step function. Thus, it can only draw 1 straight hyperplane across feature space.
- Example: For logical AND gate, positive point $(1, 1)$ can be easily partitioned from negative points $(0, 0)$, $(0, 1)$, $(1, 0)$ using a single straight line.
- Counterexample: The XOR logic gate produces point where classes are positioned at opposite diagonal corners of a square (Class 0 at $(0, 0)$, $(1, 1)$; Class 1 at $(0, 1)$, $(1, 0)$). It is mathematically impossible to draw a single straight line that separates these 2 diagonals. Overcoming this requires a Multilayer Perceptron (MLP), where hidden layer acts as nonlinear feature transform to help the space into a linearly separable one before classification.

Q3: Perceptron vs Delta Rule

Question: Contrast the error signals used in the discrete Perceptron learning rule vs the Delta rule

Answer:

- The two rule diverge fundamentally in how they measure and optimize
- The discrete Perceptron rules update weight based on the step function prediction error, $(y - \text{Sign}(w^T x))$. It updates only when a hard misclassification occurs.
- The Delta rule, perform continuous gradient descent on MSE surface using the unsaturated linear activation $(y - w^T x)$

Example: If a target is $y = 1$ and the current linear prediction is $w^T x = 0.1$, the Delta rule compute continuous gradient and update weight to push the prediction closer to exactly 1.0, minimizing the distance

Counterexample: In the same scenario ($w^T x = 0.1$), the Perceptron already classifies it correctly as positive ($\hat{y} = 1$). The error is exactly 0, so the perceptron makes absolutely no update. It ignores the fact that the margin is precariously thin (0.1), leaving the model highly vulnerable to slight data variation.

Q4 Backpropagation chain rule foundation

Question: In a 2-layer multi-layer Perceptron (MLP) state the chain rule sequence required to compute the gradient of the loss L with respect to a weight in first hidden layer ($W^{(1)}$)

Answer:

- To update hidden weight, the error signal must propagate backward from the output layer since hidden node lack explicit target value

- The chain rule expansion is:

$$\frac{\partial L}{\partial W^{(1)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial h} \cdot \frac{\partial h}{\partial z^{(1)}} = \frac{\partial z^{(1)}}{\partial W^{(1)}}$$

where z represents the pre-activation linear output and h represents the post-activation hidden state

- Example: if the hidden layer use a ReLU activation ($h = \max(0, z)$), its local derivative $\frac{\partial h}{\partial z}$ is exactly 1 for positive inputs. This allow the gradient to flow back ward efficiently without decaying, ensuring robust weight update for $W^{(1)}$

- Counterexample: The local If the network were

